

Department of Computer Science and Engineering

1. Subject Code: **SWE-502** Course Title: **Fundamentals of Software Engineering II**

2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks

3. **Prerequisites**

- SWE-401 (Fundamentals of Software Engineering I)
- CPT-301 (Programming Methodologies)

4. **1. Course Description**

This course converts the single-process application built in SWE-401 into a distributed system. Students move beyond REST into binary and graph protocols, introduce asynchronous event streaming and change data capture, harden identity with passwordless and policy-driven authorisation, impose architectural boundaries through Domain-Driven Design, and finally orchestrate and ship the result through Kubernetes and automated CI/CD. The emphasis throughout is on the failure modes that only appear once a system is split: partial failure, duplicate delivery, eventual consistency, and deployment risk.

2. Course Objectives

The primary objectives of this course are to:

- Transition students from monolithic application development to distributed systems engineering.
- Introduce advanced communication protocols and evaluate their performance implications.
- Provide hands-on experience with asynchronous, event-driven architectures and message brokers.
- Establish foundational knowledge in zero-trust security models and policy-as-code implementations.
- Cultivate rigorous DevOps practices, including container orchestration and automated software delivery.

3. Course Outcomes

After successful completion of the course, students will be able to:

- CO1.** Design and implement high-throughput multi-paradigm APIs using gRPC and GraphQL.
- CO2.** Scale applications using asynchronous event streaming (Kafka) and distributed caching (Redis).
- CO3.** Architect zero-trust security flows using OAuth 2.1, Passkeys (WebAuthn), and policy engines.
- CO4.** Orchestrate microservices and manage production deployments using Kubernetes and CI/CD pipelines.
- CO5.** Implement enterprise architecture patterns including Modular Monoliths and Domain-Driven Design.

4. Detailed Syllabus

Unit	Contents	Hrs
1	Advanced API Paradigms & High-Performance Protocols <i>Module A: Binary Protocols and gRPC</i> Protocol Buffers (proto3) schema design and evolution. gRPC unary, server-streaming, client-streaming and bidirectional modes. HTTP/2 multiplexing for low-latency service communication. Deadlines, cancellation and interceptors. <i>Module B: GraphQL at Scale</i> Schema design, resolvers and schema stitching/federation. The N+1 query problem and DataLoader batching. Query cost analysis, depth limiting and persisted queries. <i>Module C: Traffic Control & Correctness</i> Distributed rate limiting with the Token Bucket algorithm implemented via Redis Lua scripts. API idempotency keys, retries with exponential backoff and jitter, and timeout budgets. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Build a high-throughput micro-gateway that translates a REST request into a gRPC backend call while enforcing distributed rate limits using Redis. • Benchmark REST versus gRPC versus GraphQL for the same payload and report latency percentiles. • Implement idempotent write endpoints and prove correctness under duplicated requests.	30
2	Event-Driven Architectures, Messaging & Change Data Capture <i>Module A: Event Streaming with Kafka</i> Topics, partitions, offsets, consumer groups and rebalancing. Partition keys and ordering guarantees. Retention, compaction and the log as a source of truth. <i>Module B: Delivery Semantics & Patterns</i> At-most-once, at-least-once and effectively-once processing. Idempotent consumers, the outbox pattern, dead-letter topics, and schema registries with Avro/Protobuf. <i>Module C: Change Data Capture</i> CDC with Debezium: streaming PostgreSQL write-ahead-log mutations directly into event brokers. Snapshotting, backfills, and building read models from streams. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Set up a Debezium and Kafka pipeline that captures database row updates in real time and streams them to an analytical consumer service. • Implement the transactional outbox pattern and demonstrate no lost events under crash injection. • Add Redis as a distributed cache with a documented invalidation strategy.	30

Unit	Contents	Hrs
3	Zero-Trust Identity, Passkeys & Modern Authorisation <i>Module A: OAuth 2.1 and OIDC</i> Authorisation code flow with PKCE. OpenID Connect identity tokens, discovery and JWKS. Token rotation, revocation, and audience/scope design. <i>Module B: Passwordless Authentication</i> FIDO2 and WebAuthn passkeys: registration and assertion ceremonies, attestation, credential storage, and account-recovery design. <i>Module C: Fine-Grained Authorisation</i> Attribute-based access control. Policy-as-code with Open Policy Agent and Rego. Policy decision versus enforcement points, and service-to-service identity. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Implement a production-grade authentication microservice supporting WebAuthn passkeys and attribute-based access control via OPA policies. • Integrate an external identity provider using the authorisation code flow with PKCE. • Write and unit-test Rego policies covering tenant isolation.	30
4	Modular Monoliths, Clean Architecture & Domain-Driven Design <i>Module A: Strategic DDD</i> Ubiquitous language, bounded contexts, context mapping, and anti-corruption layers. Deciding what should and should not become a separate service. <i>Module B: Tactical DDD</i> Entities, value objects, aggregates and invariants. Domain events and application services. Repository abstractions and hexagonal/clean architecture layering. <i>Module C: Enforcing Boundaries</i> Spring Modulith patterns for compile-time module isolation. Architecture tests (ArchUnit), dependency rules, and preventing spaghetti dependencies in large codebases. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Refactor a tightly-coupled backend codebase into a structured Modular Monolith with strict compile-time boundary enforcement. • Produce a context map and event-storming artefact for the course domain. • Add automated architecture tests that fail the build on boundary violations.	30

Unit	Contents	Hrs
5	Advanced DevOps, Kubernetes Orchestration & CI/CD Pipelines <i>Module A: Kubernetes Fundamentals</i> Pods, ReplicaSets, Deployments, Services and Ingress. ConfigMaps and Secrets. Resource requests/limits, probes, and rolling update strategy. <i>Module B: Infrastructure as Code & Delivery</i> Declarative manifests, Helm packaging, and Infrastructure-as-Code basics. Automated CI/CD workflows with GitHub Actions: build, test, scan, containerise, deploy. <i>Module C: Operating the Deployment</i> Centralised log aggregation, basic metrics and dashboards, environment promotion, and rollback procedures. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Write a complete GitHub Actions CI/CD pipeline that builds, tests, containerises and deploys the course backend into a local Kubernetes (Minikube) cluster. • Configure liveness/readiness probes and demonstrate a zero-downtime rolling update. • Aggregate logs from all services and trace a single request across them.	30
Total Instructional Effort		150

5. Recommended Resources

- Kleppmann, *Designing Data-Intensive Applications*.
- Newman, *Building Microservices*, 2nd ed.
- Evans, *Domain-Driven Design*; Vernon, *Implementing Domain-Driven Design*.
- Official gRPC, GraphQL, Apache Kafka, Debezium, OPA and Kubernetes documentation.
- Narayanan et al., WebAuthn Level 3 specification (W3C).